

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной инженерии и Компьютерной техники

Лабораторная работа №5
Асинхронный обмен данными с ВУ
Вариант 6502

Выполнила:

Косогова Мария Александровна

Группа: Р3106

Проверила:

Ткешелашвили Нино Мерабиевна,

Преподаватель практики

Санкт-Петербург 2025

Содержание

ЗАДАНИЕ	3
ОСНОВНЫЕ ЭТАПЫ ВЫПОЛНЕНИЯ	4
1. ТЕКСТ ИСХОДНОЙ ПРОГРАММЫ	4
2. РЕАЛИЗАЦИЯ ПРОГРАММЫ НА АССЕМБЛЕРЕ	5
3. ОПИСАНИЕ ПРОГРАММЫ	6
4. ОБЛАСТЬ ПРЕДСТАВЛЕНИЯ	6
5. ОБЛАСТЬ ДОПУСТИМЫХ ЗНАЧЕНИЙ	6
6. КОДИРОВКА WINDOWS-1251, UTF-8, UTF-16	6
7. ТРАССИРОВКА	7
ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ	8
ВЫВОД	12

Задание

По выданному преподавателем варианту разработать программу асинхронного обмена данными с внешним устройством. При помощи программы осуществить ввод или вывод информации, используя в качестве подтверждения данных сигнал (кнопку) готовности ВУ.

1. Программа осуществляет асинхронный ввод данных с ВУ-2
2. Программа начинается с адреса $4CD_{16}$. Размещаемая строка находится по адресу $5E6_{16}$.
3. Строка должна быть представлена в кодировке Windows-1251.
4. Формат представления строки в памяти: АДР1: СИМВ1 СИМВ2 АДР2: СИМВ3 СИМВ4 ... СТОП_СИМВ.
5. Ввод или вывод строки должен быть завершён по символу с кодом 0A (NL). Стоп символ является обычным символом строки и подчиняется тем же правилам расположения в памяти что и другие символы строки.

Дополнительное задание

Калькулятор. Ввод цифр с ВУ-9 (цифровая клавиатура), вывод результата на ВУ-7 (семисегментный индикатор), реализовать сложение и вычитание знаковых чисел от 1 до 3-х знаков.

Основные этапы выполнения

1. Текст исходной программы

Адрес	Код команды	Мнемоника	Описание
4CA	056E	ADR_RES	Ячейка, хранящая адрес для записи символов
4CB	0040	READY	Константа для проверки готовности чтения ввода
4CC	000A	END_CHAR	Стоп - символ
4CD	0200	CLA	Очистка аккумулятора
4CE	1205	IN 5	«Spin – loop» по флагу готов ВУ- 2 Ввод содержимого SR ВУ2 в 6й бит АС Условие «6й бит АС == 1» Переход на 4CE, если Z == 1
4CF	2EFB	AND 4CB	
4D0	F0FD	BEQ 4CE	
4D1	1204	IN 4	Ввод символа
4D2	7EF9	CMP 4CC	Выставить флаги по результату АС – 000A
4D3	F00B	BEQ 4DF	Если Z == 1, переход к IP = 4DF
4D4	0680	SWAB	Обмен старшего и младшего байтов
4D5	E8F4	ST IP – 12	Косвенное относительное сохранение
4D6	1205	IN 5	«Spin – loop» по флагу готов ВУ- 2 Ввод содержимого SR ВУ2 в 6й бит АС Условие «6й бит АС == 1» Переход на 4CE, если Z == 1
4D7	2EF3	AND 4CB	
4D8	F0FD	BEQ 4D6	
4D9	1204	IN 4	Ввод символа
4DA	7EF1	CMP	Выставить флаги по результату АС – 000A
4DB	F006	BEQ 4E2	Если Z == 1, переход к IP = 4E2
4DC	48ED	ADD (IP + 19)	Косвенное относительное сложение
4DD	EAEC	ST (IP - 20)+	Косвенное автоинкрементное сохранение
4DE	CEEF	JUMP 4CE	Прямой относительный переход к 4CE
4DF	0680	SWAB	Обмен старшего и младшего байтов
4E0	E8E9	ST(IP + 23)	Косвенное относительное сохранение
4E1	0100	HLT	Остановка
4E2	48E7	ADD (IP + 25)	Косвенное относительное сложение
4E3	E8E6	ST (IP + 26)	Косвенное относительное сохранение
4E4	0100	HLT	Остановка
05E6	D1C8	Слово «СИРЕНЬ» в кодировке Windows-1251 1101 0001 1100 1000 1101 0000 1100 0101 1100 1101 1101 1100	
05E7	D0C5		
05E8	CDDC		
0570	0A00	Стоп - символ	

2. Реализация программы на ассемблере

```
ORG 0x4CA
ADR_RES: WORD $RES
READY: WORD 0x40
END_CHAR: WORD 0x0A

START: CLA

SYMBOL1:  IN 5
           AND READY
           BEQ SYMBOL1
           IN 4
           CMP END_CHAR
           BEQ EXIT_ODD
           SWAB
           ST (ADR_RES)

SYMBOL2:  IN 5
           AND READY
           BEQ SYMBOL2
           IN 4
           CMP END_CHAR
           BEQ EXIT_EVEN
           ADD (ADR_RES)
           ST (ADR_RES)+
           JUMP SYMBOL1

EXIT_ODD: SWAB
           ST (ADR_RES)
           HLT

EXIT_EVEN: ADD (ADR_RES)
           ST (ADR_RES)
           HLT

ORG 0x5E6
RES: WORD 0x00
```

3. Описание программы

Программа осуществляет асинхронный ввод данных с ВУ-2

4. Область представления

- ADR_RES – 11-ти разрядное беззнаковое число, адрес начала хранения символов
- READY – 16-ти разрядная константа
- RES – 16-ти разрядная ячейка для хранения символов, где старший байт – код первого символа, а младший байт – код второго символа

5. Область допустимых значений

- $[56E, 7FF] \cup [0, 4C9]$ – 16-ти разрядные беззнаковые числа, которые хранят в себе по 2 символа
- $[21, FF]$ – введенный символ
- Максимальное возможное количество символов для ввода
 $7FF - 56E = 2047 - 1390 = 657$ – количество ячеек до 7FF
 $4C9 = 1225$ – количество ячеек от 0 до 4C9
 $1225 + 657 = 1882$ – всего ячеек
 $1882 * 2 = 3764$ – максимальное количество символ без учета стоп-символа (так как 8 бит занимает 1 символ, а в одной ячейке у нас хранятся 2 символа $\Rightarrow$ 16 бит)
 $3764 - 1 = 3763$ - максимальное количество символ с учетом стоп-символа (занимает 8 бит)

6. Кодировка Windows-1251, UTF-8, UTF-16

Слово «СИРЕНЬ»

- Кодировка Windows-1251
С И Р Е Н Ь
D1 C8 D0 C5 CD DC
- Кодировка UTF-8
С И Р Е Н Ь
D0A1 D098 D0A0 D095 D09D D0AC
- Кодировка UTF-16
С И Р Е Н Ь
0421 0418 0420 0415 041D 042C

7. Трассировка

Адрес	Код	IP	CR	AR	DR	SP	BR	AC	PS	NZVC	Адрес	Новый код
4CD	0200	4CE	0200	4CD	0200	000	04CD	0000	004	0100		
4CE	1205	4CF	1205	4CE	1205	000	04CE	0040	004	0100		
4CF	2EFB	4D0	2EFB	4CB	0040	000	FFFB	0040	000	0000		
4D0	F0FD	4D1	F0FD	4D0	F0FD	000	04D0	0040	000	0000		
4D1	1204	4D2	1204	4D1	1204	000	04D1	00D1	000	0000		
4D2	7EF9	4D3	7EF9	4CC	000A	000	FFF9	00D1	001	0001		
4D3	F00B	4D4	F00B	4D3	F00B	000	04D3	00D1	001	0001		
4D4	0680	4D5	0680	4D4	0680	000	04D4	D100	009	1001		
4D5	E8F4	4D6	E8F4	5E6	D100	000	FFF4	D100	009	1001	5E6	D100
4D6	1205	4D7	1205	4D6	1205	000	04D6	0040	009	1001		
4D7	2EF3	4D8	2EF3	4CB	0040	000	FFF3	0040	001	1001		
4D8	F0FD	4D9	F0FD	4D8	F0FD	000	04D8	0040	001	0001		
4D9	1204	4DA	1204	4D9	1204	000	04D9	00C8	001	0001		
4DA	7EF1	4DB	7EF1	4CC	000A	000	FFF1	00C8	001	0001		
4DB	F006	4DC	F006	4D8	F006	000	04DB	00C8	001	0001		
4DC	48ED	4DD	48ED	5E6	D100	000	FFED	D1C8	008	1000		
4DD	EAEC	4DE	EAEC	5E6	D1C8	000	FFEC	D1C8	008	1000	5E6	D1C8
4DE	CEEF	4CE	CEEF	4DE	04CE	000	FFEF	D1C8	008	1000		

Дополнительное задание

Калькулятор. Ввод цифр с ВУ-9 (цифровая клавиатура), вывод результата на ВУ-7 (семисегментный индикатор), реализовать сложение и вычитание знаковых чисел от 1 до 3-х знаков.

ORG 0x000

P1: WORD \$MASSIVE_NUM1 ; 000

P2: WORD \$MASSIVE_NUM2

MASSIVE_NUM1: WORD 0x0F, 0x0F, 0x0F ; 002 003 004

MASSIVE_NUM2: WORD 0x0F, 0x0F, 0x0F ; 005 006 007

RES: WORD 0x00, 0x00, 0x00, 0x00 ; 008 009 00a 00b

T: WORD 0x00 ; 00c

ADD_NUMBERS: LD -(NUM1END)

ADD -(NUM2END)

ADD -(ADR_RES)

CMP #0x0A

BGE CONTIN_ADD ; если сумма больше 10 - переходим

ST (ADR_RES) ; иначе меньше - сохраняем в результат

JUMP CHECK_ADD

CONTIN_ADD: SUB #0x0A

ST (ADR_RES)

LD #0x01

ST -(ADR_RES) ; запись переноса

LD (ADR_RES)+

CHECK_ADD: LD NUM1END

CMP #0x02

BEQ OUTPUT

JUMP ADD_NUMBERS

SUB_NUMBERS: LD -(NUM1END)

SUB -(NUM2END)

BGE CHECK_SUB

LD -(NUM1END)

SUB #0x01

ST (NUM1END)+

LD (NUM1END)

ADD #0x0A

SUB (NUM1END)

CHECK_SUB: ST -(ADR_RES)

LD ADR_RES

CMP #0x09

BEQ OUTPUT

BGE SUB_NUMBERS

JUMP OUTPUT

OUTPUT: LD #000C ; сброс счетчика R

ST \$ADR_RES

WRITELOOP:

LD -(ADR_RES)

ADD \$T

OUT 0x14

LD \$T

```

        ADD #0x10
        ST $T
        LD $ADR_RES
        CMP #0x08
        BEQ SIGNWR
        JUMP WRITELOOP
SIGNWR: LD $SIGN_RES      ; пишем знак R
        BEQ ENDWR
        LD #0xA
        ADD $T
        OUT 0x14
        HLT
ENDWR:  LD #0xF
        ADD $T
        OUT 0x14
        HLT

```

```

POINTER1: WORD $MASSIVE_NUM1 ; 000
POINTER2: WORD $MASSIVE_NUM2

```

```

; подпрограмма для чтения
READ: IN 0x1D
      AND #0x40
      BEQ READ
      IN 0x1C
      ST &1
      RET

```

```

SIGN_NUM1: WORD 0x00
SIGN_NUM2: WORD 0x00
SIGN_RES:  WORD 0x00
ADR_RES:   WORD 0x0C
j: JUMP ADD_NUMBERS
s: JUMP SUB_NUMBERS

```

```

START: CLA ;
      PUSH
READ_NUM1: CALL READ ; читаем
          CMP #0x0A ; минус?
          BEQ SAVE_SIGN_NUM1 ; сохраняем знак первого числа
          CMP #0x0A ; это число?
          BLT SAVE_FNUM1 ; да - сохраняем
          JUMP READ_NUM1 ; нет - вводите снова
SAVE_SIGN_NUM1: LD #1 ; флаг отрицательности числа
               ST SIGN_NUM1 ; сохраняем
               JUMP READ_NUM1 ; теперь нужна сама цифра/число
SAVE_FNUM1: ST (POINTER1)+ ; сохраняем цифру
FURTHER_READ_NUM1: CALL READ
                  CMP #0x0A ; это минус?
                  BEQ SAVE_SIGN_NUM2 ; да - знак второго числа
                  отриц
                  CMP #0x0B ; это плюс?
                  BEQ SAVE_SIGN_NUM2 ; да - знак второго числа полож
                  CMP #0x0A ; это цифра?

```

```

        BLT SAVE_NUM1 ; да - сохраняем
        JUMP FURTHER_READ_NUM1
SAVE_NUM1: ST (POINTER1)+ ; да - сохраняем
        LD POINTER1 ; загружаем значение счетчика
        CMP #0x05 ; вышел за рамки?
        BEQ INPUT_SIGN ; да - вводите знак операции=знак второго числа
        JUMP FURTHER_READ_NUM1 ; нет - можно ввести еще

INPUT_SIGN: CALL READ
        CMP #0x0A ; это минус?
        BEQ SAVE_SIGNM_NUM2 ; сохраняем как знак второго числа
        CMP #0x0B ; это плюс?
        BEQ SAVE_SIGN_NUM2 ; переходим к обработке
        JUMP INPUT_SIGN

SAVE_SIGNM_NUM2: LD #1
                ST SIGN_NUM2
                JUMP READ_NUM2

NUM1END: WORD 0x05
NUM2END: WORD 0x08
NUM1BEGIN: WORD 0x02
NUM2BEGIN: WORD 0x05

SAVE_SIGN_NUM2: LD #0
                ST SIGN_NUM2

READ_NUM2: CALL READ
        CMP #0x0A ; это число?
        BLT SAVE_NUM2 ; да - сохраняем
        JUMP READ_NUM2 ; нет - вводите заново
SAVE_NUM2: ST (POINTER2)+
        LD POINTER2
        CMP #0x08
        BEQ INPUT_EQUAL
FURTHER_READ_NUM2: CALL READ
                CMP #0x0F
                BEQ INPUT_EQUAL1
                CMP #0x0A
                BLT SAVE_NUM2
                JUMP FURTHER_READ_NUM2

INPUT_EQUAL: CALL READ
INPUT_EQUAL1: CMP #0x0F
                BEQ TRANSFORM
                JUMP INPUT_EQUAL

TRANSFORM: CALL NORM1
                CALL NORM2
                LD SIGN_NUM1
                CMP SIGN_NUM2
                BEQ IFSAME
                JUMP IFDIFFERENT

```

```

IFSAME: LD SIGN_NUM1
        BEQ j ;ADD_NUMBERS ; если знаки + => add
        LD #0x01
        ST SIGN_RES
        JUMP j ;ADD_NUMBERS

IFDIFFERENT: LD (NUM1BEGIN)+ ; ЗАГРУЖАЮ РАЗРЯДЫ ЧИСЛА1
              SUB (NUM1BEGIN)+ ; вычитаю РАЗРЯДЫ ЧИСЛА2
              BEQ IFDIFFERENT ; равны загружаем след разряды
              BLT NUM2_MORE
              LD SIGN_NUM1
              ST SIGN_RES
              JUMP SUB_NUMBERS

NUM2_MORE: LD SIGN_NUM2
              ST SIGN_RES
              LD #0x08
              ST NUM1END
              LD #0x05
              ST NUM2END
              JUMP s ;SUB_NUMBERS

NORM1: LD 0x04
        CMP #0x0F
        BEQ NORMALZ1
        RET
NORMALZ1: LD 0x03
              ST 0x04
              LD 0x02
              ST 0x03
              CLA
              ST 0x02
              JUMP NORM1

NORM2: LD 0x07
        CMP #0x0F
        BEQ NORMALZ2
        RET
NORMALZ2: LD 0x06
              ST 0x07
              LD 0x05
              ST 0x06
              CLA
              ST 0x05
JUMP NORM2

```

Вывод

Выполнение лабораторной работы по ОПД позволило мне более углубленно изучить БЭВМ. Данная работа дала мне возможность применить полученные теоретические знания на практике, развить навыки работы с информацией. В процессе выполнения я научилась работать с асинхронным вводом-выводом данных. Приобретённый опыт поможет мне при изучении последующих тем по предмету ОПД.